

Monday Range Bot Roadmap

From cleaned-up backtest to paper trading, live execution, and eventually a modular multi-strategy crypto trading system

This roadmap captures the project direction from the current state of the repository to the eventual target: a fully automated but risk-controlled trading system. The immediate priority is not to rush into live orders. The priority is to make the strategy engine, data pipeline, backtest, paper trader, dashboard, and risk controls trustworthy enough that live trading becomes a small final adapter rather than a dangerous rewrite.

Current state snapshot

Area	Status
Backtest	Existing Monday range logic, now aligned with paper-trader entry timing, Friday cutoff, fees/slippage intent, and TP2 configurability.
Paper trading	Paper-only engine exists. It fetches public Binance candles, uses completed candles, persists state, logs events, and avoids historical fake trades in forward mode.
Correctness tests	Parity tests exist between backtest and paper backfill. Execution helper tests exist. Current test suite is passing.
Shared execution helpers	Slippage, fees, TP levels, stop/TP hit checks, and Friday cutoff detection have been extracted into shared pure helpers.
Live trading	Not implemented yet, deliberately. This is correct at the current stage.

The high-level architecture you are aiming for

The mature system should separate research, execution, monitoring, and risk. The strategy should not know whether it is being backtested, paper traded, or traded live. It should emit intended actions. A broker layer should decide whether those actions are simulated or sent to Binance. A risk layer should be able to block any order before it reaches the broker.

Layer	Purpose	Examples
Data layer	Fetch, cache, validate, and serve candles.	Binance klines, local Parquet/CSV cache, missing candle checks, UTC normalization.
Feature layer	Compute reusable market features.	Monday range high/low/mid, volatility regime, ATR, trend filters, market condition labels.
Strategy layer	Convert features into signals.	Monday range breakout/fade, later momentum, mean reversion, volatility breakout, regime-specific strategies.
Execution simulation	Apply fills, slippage, fees, stops, targets, and exits.	Shared execution helpers, backtest engine, paper engine.
Broker layer	Abstract paper vs live trading.	PaperBroker first, BinanceSpotBroker later. Same interface.
Risk layer	Final safety gate before orders.	Max order size, max daily loss, max weekly loss, kill switch, symbol whitelist.
Monitoring layer	Make the system observable.	Streamlit dashboard, trade journal, data health, alerts.

Roadmap gates

Each gate should have a clear pass condition before you move on. This prevents the classic failure mode where a bot gets more complicated before it is correct.

Gate	Goal	Main tasks	Pass condition
0. Foundation already done	Make the single-strategy engine sane.	Binance candles, forward paper mode, completed candles only, backtest/paper parity, execution helpers.	Tests pass and no known P0 logic mismatch remains.
1. Read-only dashboard	Make the bot observable.	Streamlit dashboard reading paper_state.json, paper_trades.csv, candle cache. Overview, strategy state, trade journal, performance, data health, config.	Dashboard runs locally and never writes orders/state/logs.
2. Scheduled forward paper trading	Run the bot like it is live, but with fake execution.	Hourly runner, state persistence, stale data detection, log rotation, restart safety, alerts.	Runs for weeks without duplicate trades, missing logs, or state corruption.
3. Research upgrade	Decide whether the strategy is worth trading.	Longer Binance history, train/validation/test split, walk-forward tests, parameter sweeps, buy-and-hold benchmark, risk metrics.	Strategy survives out-of-sample and walk-forward checks without obvious overfit.
4. Broker abstraction	Make live trading a replaceable adapter.	Define Broker interface: balances, positions, place_order, cancel_order, order status. Implement PaperBroker first.	Paper trading still works through the broker abstraction.
5. Risk engine	Ensure bad orders are blocked before execution.	Symbol whitelist, max order size, max daily loss, max weekly loss, max open exposure, kill switch, live disabled default.	Every order request passes through risk checks; tests cover blocked orders.
6. Binance live spot adapter	Allow tiny live spot orders only.	API-key handling, testnet/sandbox if useful, order placement, order status reconciliation, error handling.	Live adapter exists but defaults disabled; dry-run and tiny-size tests work.
7. Tiny live pilot	Validate real execution with negligible money.	Spot only, no leverage, tiny order size, manual review, full logging, alerts on every order.	First 10-20 live trades match expected behaviour and no accounting discrepancies appear.
8. Modular multi-strategy system	Let multiple strategies share data and trade concurrently under a portfolio risk layer.	Strategy interface, shared data bus, strategy registry, capital allocation, conflict resolution, regime routing.	Multiple strategies can run in paper mode simultaneously without conflicting state or risk limits.

Immediate next steps from today

This is the practical near-term sequence I would follow before thinking about live trading.

1. Add the read-only dashboard

Build dashboard.py and small data-loading helpers. It should read local files only and show PAPER ONLY / LIVE TRADING DISABLED. Do not allow the dashboard to place orders or edit config yet.

2. Reset and verify forward paper mode

Delete paper state/logs, run once to initialise, run again to confirm no duplicate events. Then let it run only on new completed candles.

3. Add an hourly runner

Run the paper trader a few minutes after every 1h candle closes. Start simple: cron, launchd, or a small Python loop. Log every run result.

4. Add alerts

Telegram/Discord/email alerts for entry, TP1, TP2, stop, Friday exit, stale data, exceptions, and restart events.

5. Improve research metrics

Add robust summaries: CAGR/annualized return, Sharpe, Sortino, max drawdown, win rate, profit factor, exposure time, fees paid, monthly/yearly breakdowns.

6. Run longer backtests

Pull BTCUSDT and ETHUSDT 1h/4h data across several years. Compare to buy-and-hold and cash. Use train/validation/test or walk-forward splits.

7. Decide whether the strategy deserves live pilot

Only continue if paper trading and out-of-sample research remain coherent after weeks, not after one good backtest.

Suggested project modules

Module/file	Purpose
src/binance_data.py	Public Binance data download, pagination, caching, UTC normalization.
src/features.py	Monday range and other reusable feature calculations.
src/signals.py	Strategy signal logic independent of execution venue.
src/execution_helpers.py	Pure shared helpers for slippage, fees, targets, hit checks, and Friday cutoff.
src/backtest.py	Historical simulation using shared helpers.
src/paper_trader.py	Forward paper trading state machine and event logging.
src/dashboard_data.py	Read-only loading/normalization functions for dashboard files.
dashboard.py	Streamlit monitoring app. Read-only.
src/brokers/base.py	Future Broker interface.
src/brokers/paper.py	Future PaperBroker implementation.
src/brokers/binance_spot.py	Future Binance live spot adapter, disabled by default.
src/risk.py	Future risk checks and kill switch.
src/strategies/	Future modular strategy classes. Monday range becomes one strategy among many.

Eventual multi-strategy architecture

The long-term design is not one giant bot. It is a platform where multiple strategies share the same clean data and execution/risk infrastructure.

Core idea

- One shared data layer fetches and validates market data once.
- Each strategy reads the same data/features but emits its own proposed orders.
- A portfolio/risk layer decides whether the combined set of orders is allowed.
- A broker layer executes either in paper mode or live mode.
- The dashboard monitors the whole system, not just one strategy.

Component	Future role
Strategy interface	Every strategy implements a common API such as <code>generate_signals(context)</code> or <code>propose_orders(context)</code> .
Strategy registry	Enable/disable strategies from config without changing core engine code.
Shared feature store	Precompute features such as volatility, trend, range width, ATR, volume regime, and market condition labels.
Regime router	Run or weight strategies based on market condition: trending, ranging, high volatility, low volatility, breakout regime, etc.
Portfolio allocator	Assign capital/risk budgets across strategies and symbols.
Conflict resolver	Handle cases where one strategy wants long BTC and another wants short BTC.
Global risk engine	Limit total exposure, daily loss, weekly loss, correlated positions, and order frequency.

Live trading safety checklist

- Use spot only at first. No leverage, futures, margin, martingale, or grid averaging.
- Live trading must be disabled by default in config.
- Require explicit environment variables for API keys and live mode.
- Start with tiny order size, e.g. the smallest practical USDT amount.
- Implement max order size, max daily loss, max weekly loss, and a manual kill switch.
- Log every order request, exchange response, fill, balance check, and error.
- Reconcile local state against exchange balances after every run.
- Alert on every live order and every failed API call.
- Run several weeks of forward paper trading before first live order.

Definition of done for v1

Version 1 is not "profitable forever". Version 1 is a trustworthy single-strategy system that can run unattended in paper mode and, later, place tiny live spot orders safely.

- Backtest and paper-trading behaviours are aligned and covered by parity tests.
- Paper trader runs on schedule and survives restarts without duplicate events.
- Dashboard shows current state, trades, performance, and data health.
- Alerts report entries, exits, stale data, and errors.
- Risk engine blocks oversized or unsafe orders.

- Live adapter is isolated behind a broker interface and disabled by default.
- First live pilot uses tiny spot-only order sizes with full logging and manual review.

Guiding principle: build a reliable trading system first, and only then decide whether this particular strategy deserves real capital. Even if the strategy is not profitable, the infrastructure is valuable as a quant/software project and as the base for future modular strategies.